# Selectivity Estimation for Relation-Tree Joins

Chao Zhang
University of Helsinki, Finland
Renmin University of China, China
chao.z.zhang@helsinki.fi

Jiaheng Lu
University of Helsinki, Finland
jiaheng.lu@helsinki.fi

## ABSTRACT

Estimating the join selectivity is a crucial problem in many aspects of query processing, such as query optimization and query refinement. Selectivity estimation has been extensively studied for the relational joins in SQL queries and structural joins in path-oriented queries. However, as leading databases have supported the multi-model data management on relational and tree-structured data together, a new problem has arisen: the existing estimation techniques mainly work for a single model but not for the heterogeneous situation due to the cross-model joins. A straightforward combination of existing estimators cannot provide a satisfactory estimation quality.

This paper studies the problem of selectivity estimation for cross-model joins with relational and tree-structured data. Our estimator is based on the Kernel Density Estimation (KDE) model, which is a statistical approach using a data sample to approximate multivariate probability distribution. KDE has been successfully applied in relational databases to estimate the selectivity of range and join query. In this work, we propose an estimation method called location-value estimation (LVE) model based on KDE, which considers both value joins and structural joins in relational and tree-structured data. To boost the estimation efficiency in large data samples, we further propose the max-min approximation (MMA) and grid-based approximation (GBA) models to approximate the KDE contribution. Extensive experiments on four real and synthetic datasets demonstrate the effectiveness, efficiency, and scalability of our techniques.

## CCS CONCEPTS

• **Information systems** → *Query optimization*; *Federated databases.*

## KEYWORDS

KDE, Selectivity Estimation, Relation-Tree Join

**ACM Reference Format:**
Chao Zhang and Jiaheng Lu. 2020. Selectivity Estimation for Relation-Tree Joins. In *32nd International Conference on Scientific and Statistical Database Management (SSDBM 2020), July 7–9, 2020, Vienna, Austria.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3400903.3400921

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
*SSDBM 2020, July 7–9, 2020, Vienna, Austria*
© 2020 Association for Computing Machinery.
ACM ISBN 978-1-4503-8814-6/20/07…$15.00
https://doi.org/10.1145/3400903.3400921

A relation-tree join involving a customer table and a transaction JSON document

(a) Customer table

| id | name | age | gender |
|---|---|---|---|
| c1 | Rafael Oliverira | 21 | female |
| c2 | Chris Jasani | 53 | female |
| c3 | Jerzy Ciesla | 22 | female |
| c4 | Mahinda Perera | 25 | male |
| c5 | Albade Maazou | 41 | male |
| c6 | David Baker | 31 | male |

(b) Transaction document

```
[
 {"customer":{"id":"c1",
              "itemlist":[
               {"item":"Molten Ball 6"}]}},
 {"customer":{"id":"c6",
              "itemlist":[
               {"item":"Spading Ball 7"},
               {"item":"REVIVL Bump"}]}}
]
```

(c) PostgreSQL Query:

```
SELECT Rc.id, Rc.name, Tc->'itemlist'
FROM jsonb_array_elements(Transaction->'customer') Tc
INNER JOIN Customer Rc
ON Rc.id = Tc->>'id'
```

**Figure 1: The motivating example of a relation-tree join involving a customer table and a transaction JSON document. Below the datasets is the join query written in SQL query with JSON path expressions of the PostgreSQL database.**

## 1 INTRODUCTION

As data volume and variety increase at an ever-faster rate, the trend towards utilizing a unified platform to manage the relational and tree data together accelerates [19, 38]. On one hand, all leading RDBMS platforms have supported tree data management using the SQL-extension query languages such as SQL/JSON [25] and SQL/XML [23]. On the other hand, emerging multi-store systems [3] and multi-model databases [20, 21, 38, 39] have developed new query languages to query the tabular and tree data simultaneously.

Example 1. *Given a customer table and a transaction document shown in Figure 1, which are stored in a PostgreSQL database as a relational table and a JSON document,respectively. A meaningful query here is to find the customers' profile information from the table with their purchased items from the document for the business analysis on customers' purchase behaviors. Particularly, we implement the query using SQL with path-oriented expressions for JSON (see Figure 1c). Note that this query is achieved by joining the tuples in $R_c$ with an array of subtrees $T_c$ on the common key id. We term this kind of join as a relation-tree join.*

By leveraging the relation-tree join, one can easily ingest and analyze heterogeneous data in real time without involving any process of extraction, transformation, and loading (ETL). In addition to the e-commerce scenario we considered in the example, the relation-tree join can have a larger number of other potential applications, ranging from integrated health-care services [8], online recommender system [29] to the smart traffic management [27].

Accurate estimates for the join result are essential as a query optimizer can utilize them to select the optimal plan [32]. There are also other cases that the estimates of query results are useful before

evaluating the full query result. For instance, to design a benchmark query, a quick feedback about the join size of the query would help the user to decide whether to modify the query with another selection predicate or to configure other binding parameters for the same query [6, 38]. Similarly, result size estimation can be valuable for the "next-step" decision in query refinement [24].

The problem of join selectivity estimation has been extensively studied in the relational database [1, 7, 15, 16, 18, 33, 34]. Particularly, these studies can be divided into three classes. The first class is the sampling-based approach, which maintains a data sample from base tables, then estimates the join size by evaluating the join on the sample. Many sampling methods have been proposed, e.g., correlated sampling [34], end-biased [7] and index-based sampling [18], by utilising either common hash functions or indices to capture the join correlations. The second class is the sketch-based method [28] that uses a small sketch to provide approximate answers. AGMS sketch [1] is a representative of this category, which maps the distinct value in the stream to a 2-wise independent random variable, then increments the values to the counter that is used to estimate the join size. The third class is the model-based method that utilizes machine learning models to join selectivity estimation, two representative approaches are the graphical model [33] and convolutional neural network [16]. Considerable research attention has also been paid to structural join estimation for tree pattern query optimization [22, 26, 36, 37, 40], which can also be broadly divided into three classes, including the sampling-based [22, 36], histogram-based [36, 37], and sketch-based [26, 40]. Specifically, the sampling-based approaches sample subtrees to derive the approximate answers. The histogram-based approaches construct the binary relationship histograms to estimate the structural join size. The sketch-based approaches rely on tree synopses to estimate the twig query selectivity.

While a host of work has been proposed to address the problem of join selectivity estimation on either relational or tree data, it is unclear how to apply these methods to the relation-tree join selectivity estimation (RTJSE). Straightforwardly combining the single-model estimators fails to capture the join correlations among data, hence unable to obtain high-quality estimates. For example, naively computing the join contribution from the relational and tree samples often leads to a biased and error-prone estimation due to the missing of join partners in the samples. What's worse, when a relation-tree join consists of multiple cross-model value joins and a complex structural join, it is rather challenging to obtain a good estimate for small samples. Naturally, an open question arises: *Can we have a unified and effective method to address the RTJSE problem?*

Kernel Density Estimation (KDE) is a data-driven, non-parametric method to estimate probability densities based on a data sample [31]. In particular, KDE has been widely used to estimate the selectivity in the database research literature [12, 15, 41, 42]. These research efforts have shown that KDE converges faster to the underlying distribution than histogram approaches do and can consistently offer superior estimation quality than the purely sampling approach. To the best of our knowledge, the question of how to apply a KDE-based selectivity estimator to a relation-tree join has not been covered by the literature.

In this paper, we propose a new kernel-based estimation technique for the RTJSE problem. Our basic idea is to build the KDE model on the relational and tree samples, then obtain an estimator combining the value joins and structural joins. More specifically, we first build an index over the tree data to encode the structural and value information. Then, we sample the tuples from the base tables and sample the tree nodes from the tree index. Finally, we apply the location-value estimation (LVE) model to the data sample to derive a joint KDE estimator including the structural join and value join. Moreover, we propose two approximation models, max-min approximation (MMA) and grid-based approximation (GBA) models, to further improve the estimation efficiency with a guaranteed gap bound, respectively. To summarize, we have made the following contributions:

- We study a new join selectivity estimation problem for the relation-tree join, named RTJSE problem, which can serve as a building block towards relation-tree query processing in modern databases.
- We propose a new kernel-based estimation model called the Location-value estimation (LVE) to tackle the RTJSE problem. We start with a simple case of relation-tree joins. Then, we extend it to more complicated cases.
- We propose the max-min approximation (MMA) and grid-based approximation (GBA) to enhance the estimation efficiency with bounds guaranteed.
- We have conducted extensive experiments to compare the LVE model and its variants to the state-of-art methods. The results show that our KDE model significantly outperforms them in terms of accuracy, efficiency, and scalability.

The remainder of the paper is organized as follows. Section 2 introduces the preliminaries and problem definition. Section 3 presents the estimation algorithms for the RTJSE problem. Section 4 proposes two KDE approximation methods. Section 5 presents our experimental evaluation, and Section 6 presents the related work and discussions. Finally, Section 7 concludes this work.

## 2 PROBLEM DESCRIPTION & BACKGROUND

In this section, we introduce the terminologies and approaches that we will use in this paper.

### 2.1 Relation-Tree Join

A relation $R$ has a set of attributes $A = \{A_1, ., A_n\}$. Each tuple $t \in R$ is a *n-ary* tuple denoted by $t =< v_1, ., v_n >$ where $v_i$ is the value corresponding to attribute $A_i$ from the domain dom($A_i$).

A tree is a *rooted labeled tree* $T = (r, N, K, V, E, L)$, where (1) $r$ is the unique root node. (2) $N$ is a finite set of labeled nodes having at most one incoming edge. (3) $K$ is a set of keys, and $V$ is a set of values; both are from an alphabet $\Sigma$. (4) $E \subseteq N \times N$ is a set of edges, in which each edge $(n, n')$ points from a parent node $n$ to a child node $n'$ that models a parent-child (PC) relationship. (5) $L$ is a function $f : N \rightarrow K \times V$ such that for each node $n \in N$, $L(n)$ is a key-value pair $(k, v) \in K \times V$. Note that the *rooted labeled tree* is a general abstraction of tree-structured data. Thus, the tree can be acquired from a wide range of tree data, such as XML and JSON.

In this paper, we study the cross-model join between a relation and a tree. The tuples and tree nodes are connected through value joins, which require both values to be equivalent. Moreover, there is a structural join in the tree, which is a twig pattern matching. A twig

pattern is a tree with a set of branched paths. Each edge in the path denotes a binary relationship, i.e., parent-child (PC) or ancestor-descendant (AD) relationship. We define the relation-tree join result as a table that consists of a set of all combinations of tuples derived from the relational and tree data. Each tuple should satisfy the given cross-model value join and structural join predicates. In the following, we formally define the relation-tree join.

A cross-model value join operation is denoted as $R \bowtie_\theta T$ where $R$ is a relation with a set of attributes $A$; $T$ is a tree that includes a set of keys $K$; $\theta$ is the value join predicate on the intersection of $A$ and $K$, requiring that the attribute's name and value of tuples in $R$ must be equal to the key and value of tree nodes in $T$. The join result is a table, which consists of a set of unified tuples combining the involved attributes in $R$ and keys in $T$.

A structural join is a twig pattern matching $\gamma = (N, E, f)$ where $N$ is a set of labeled query nodes; $E \subseteq N \times N$ is a set of edges; $f$ is a binary function modeling the edge type between two nodes, which includes the single-child-axis (/), the branch-child-axis ([]), and the descendant-axis (//). The join result is a set of distinct twig pattern matches, requiring any binary functions in $\gamma$ is satisfied simultaneously. For example, the structural join `customer[id]//item` is a twig pattern matching with three nodes and two edges, which returns all the descendant (//) customer nodes, along with an id child ([]) and an item descendant (//). Finally, we represent each match result as a tuple. Note that although we consider only a two-level case in the example, the twig pattern may be multi-level and recursive.

DEFINITION 1 (Relation-Tree Join). *Formally, given a relation $R$ and a tree $T = (r, N, K, V, E, L)$, a relation-tree join is defined as $R \bowtie_\eta T$, where the relation-tree join predicate is $\eta = \theta \wedge \gamma$ that combines the cross-model value join predicate $\theta$ and structural join predicate $\gamma$.*

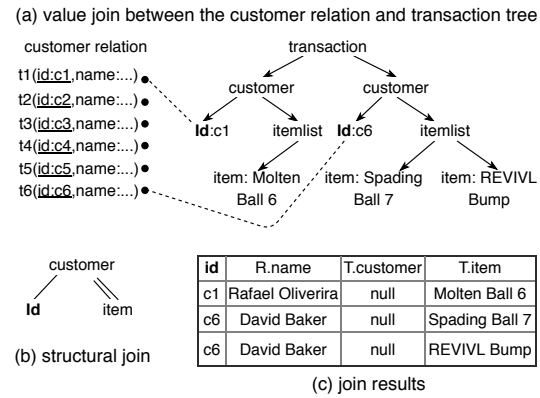
Example 2. *We use the motivation example in Figure 2 to illustrate the algebra expression for a relation-tree join. According to Definition 1, the join can be represented as follow:*

$$R_c \bowtie_{(R_c.id=T_c.id)\wedge(\exists x.k=customer, y.k=id, z.k=item, x[y]//z)} T_c \tag{1}$$

*The above join is a two-way relation-tree join that involves a customer relation $R_c$ and a transaction tree $T_c$. The relation $R_c$ has a set of attributes $A = \{id, name, age, gender\}$. The tree $T_c$ contains a set of keys $K = \{customer, id, itemlist, item\}$, and nodes of customer and itemlist have null value. Specifically, the relation-tree join consists of a cross-model value join and a structural join. The cross-model value join is on the key id that corresponds to the attribute id of relation $R_c$ and key id of nodes in tree $T_c$, respectively. The structural join is a twig pattern matching (see Figure 2b), which includes (1) a PC relationship requiring each customer node has a child of id node, and (2) an AD relationship such that each customer node has a descendant of item node. Consequently, the join results are returned as as a table (see Figure 2c), which consists of 3 joined tuples.*

## 2.2 Research Problem Description

Given a relation-tree join $Q$ over a relation $R$ and a tree $T$. Our goal is to predict $Q$'s join selectivity denoted by $\Phi(Q)$, which has the following formula:




| id | R.name | T.customer | T.item |
|---|---|---|---|
| c1 | Rafael Oliverira | null | Molten Ball 6 |
| c6 | David Baker | null | Spading Ball 7 |
| c6 | David Baker | null | REVIVL Bump |

**Figure 2: Illustration for a relation-tree join and join results.**

$$\Phi(Q) = \frac{|R'(Q) \bowtie T'(Q)|}{||R| \times |T||} \tag{2}$$

where the numerator is the relation-tree join size, and the denominator is the size of the relation-tree Cartesian product. Particularly, $|R|$ denote the tuple size of the relation $R$, and $R'(Q)$ further includes the selection and join predicates. $|T|$ denote the size of the Cartesian product of the subtree nodes, and $T'(Q)$ are the distinct matches of $Q$. Obviously, $T'(Q)$ (resp. $R'(Q)$) is a subset of $T(Q)$ (resp. $R(Q)$), thus the join selectivity $\Phi(Q)$ is between 0 and 1. Note that the size of $T(Q)$ can be different with schema information. For instance, if we know each customer node must have only one id child node, then the upper bound of the AD join is the size of customer nodes.

Example 3. *Recall the motivation example in Figure 2, given the schema information that the customer node must have only one id node, we know that both the size of tuples in relation $R$ and the size of subtrees in tree $T$ are 6. Hence, the join selectivity is $3/(6*2*3) \approx 0.083$ according to the Equation 2. Our goal is to use a small portion of data to estimate the join selectivity.*

## 2.3 Kernel Density Estimation

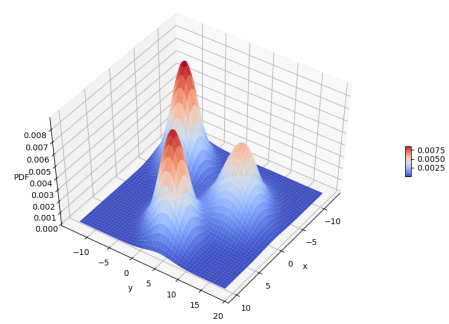



**Figure 3: Gaussian 2D KDE with 200 sample points**

Given a random sample $S = \{\overrightarrow{u}^{(1)}, \ldots, \overrightarrow{u}^{(s)}\}$ of size $s$ from a $d$-dimensional dataset $D \in \mathbb{R}^d$, the kernel estimator assigns a probability density to each point $u^{(i)} \in S$ such that $\hat{p(u)} : \mathbb{R}^d \to \mathbb{R}$. Finally, it averages the KDE contributions $\hat{p(S)}$ of sample $S$ to derive

an estimate $\Phi(S)$. Figure 3 visualizes the surface of Gaussian 2D KDE with 200 sample points. By spreading the local probability distributions of these sample points to the entire domain, it is clearly visible that the data points are clustered around three centers. The final estimate is calculated by integrating over the KDE surface. KDE model is inherently a data sample and has no any assumptions on the underlying data, thus are generally more robust than parametric techniques. KDE has been widely used to estimate the selectivity in the database research literature [12, 15, 41, 42], and has been shown to consistently offer superior estimation quality. Particularly, the KDE contribution $\hat{p(x)}$ of a point $x$ is defined as follow:

$$\hat{p(x)} = \frac{1}{S}\sum_{u\in S} K(\frac{x-u}{h}) = \frac{1}{S\cdot h}\sum_{u\in S} K_h(x-u) \tag{3}$$

where the function $K$ is the kernel function, which defines the shape of the local probability distributions. Examples of kernel functions include *Gaussian, Triangle, Epanechnikov, and Ball* [31]. In this work, we use the *Gaussian* kernel by default, which has the following form:

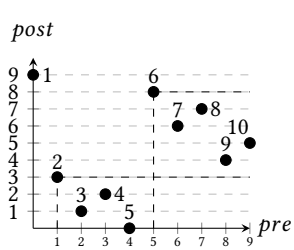
$$K(\vec{x}) = (2\pi)^{-\frac{d}{2}} e^{-\frac{1}{2}\vec{x}^T\vec{x}} \tag{4}$$

In the Equation 3, the $h$ is the smoothing factor called bandwidth which controls the scope or width of the kernel distributions. Choosing the appropriate value for bandwidth is vital for obtaining a good KDE. If the bandwidth is too small, the estimator overfits the sample. If it is too large, all local information is lost. Thus, judicious choice of bandwidth is required. A classical method is the *Scott's rule* [31], which uses the normal scale rule to approximate the true distribution by the Normal distribution. [2]. Consequently, the bandwidth $\hat{h}^{scott}$ via *Scott's rule* is defined as follow:

$$\hat{h}_i^{scott} = |s|^{-\frac{1}{d+4}} \cdot \sigma_i \tag{5}$$

where s denotes the sample size; d is dimensionality and $\sigma_i$ is the $i$-th attribute in the sample. However, there is a problem of *Scott's rule* that it often leads to overly smoothed estimators that underfit the true distribution since the real-world data is seldom normal [12]. Despite this shortcoming, most of KDE-related works [12, 15, 41] still adopt the *Scott's rule* as the bandwidth initialization due to its simplicity. Our work relies on *Scott's rule* in the first step as well.

## 2.4 A tree encoding for relation-tree joins




| | pre | post | level | (key:value) |
|---|---|---|---|---|
| 1 | 0 | 9 | 0 | Transaction:null |
| 2 | 1 | 3 | 1 | customer:null |
| 3 | 2 | 1 | 2 | id:c1 |
| 4 | 3 | 2 | 2 | itemlist:null |
| 5 | 4 | 0 | 3 | item:Molten Ball 6 |
| 6 | 5 | 8 | 1 | customer:null |
| 7 | 6 | 6 | 2 | id:c6 |
| 8 | 7 | 7 | 2 | itemlist:null |
| 9 | 8 | 4 | 3 | item:Splading Ball 7 |
| 10 | 9 | 5 | 3 | item:REVIVL Bump |

**Figure 4: Pre/post plane and the corresponding node encoding table for the tree data of Figure 1b.**

In order to estimate the structural join and value join simultaneously, we extend a previous tree encoding technique [10] to build an index upon the tree data. In a nutshell, we uses the *pre-order* and *postorder ranks* of tree nodes with the value information to map nodes onto a two-dimensional plane. Figure 4 shows the two-dimensional pre/post plane with the encoding table from the tree instance of Figure 1b. The encoding table includes the pre/post encoding, the node level, and the value information of a tree node. The pre/post encoding makes an AD relationship between node $a$ and $d$ satisfying the formula $a.pre < d.pre \wedge a.post > d.post$. The node level denotes the length of the path from the root to the node, with the level information, we can decide a PC relationship by checking if the ancestor node's level number is exactly 1 less than that of the descendant nodes. The value information records the node label, which is a key/value pair that can be joined to a relational tuple.

Once we have the tree index built, we can make use of this index to help in estimating the selectivity of a relation-tree join. Given the relation-tree join, a naive solution is to sample the tuples and tree encoding, identify the matched subtrees, then compute the join size. However, a problem is that it often results in a biased and error-prone estimation due to the sheer size of whole data, the complex tree structure, and the skewed join value distribution. In this work, we leverage the KDE model to obtain a more robust and accurate estimate.

# 3 KDE ON RELATION-TREE JOIN

In this section, we discuss how to apply KDE to the RTJSE problem. We start with the basic case of relation-tree join in Section 3.1, which involves a cross-model equijoin and a AD structural join; we then present how to apply the Location Value Estimation (LVE) model to the more complex cases in Section 3.2.

## 3.1 A Basic Relation-Tree Join

Let us first consider a relation-tree join that involves a cross-model value join and a AD structural join, the query Q can be expressed as follow:

$$Q = R \bowtie_{(R.A=T.A)\wedge(\exists x.k=A, y.k=D, A//D)} T \tag{6}$$

where A is join key for the cross-model value join between a relation $R$ and tree $T$, and nodes with key A should be the ancestors of those with key D in tree $T$.

For an individual join value $v$ and a pre/post numbering, $m$ and $n$, we can express its true join size as follow:

$$|R| \cdot p_R(A.v = v) \cdot |T_A| \cdot p_{T_A}(A.v = v \wedge A.pre < m \wedge A.post > n) \cdot |T_D| \cdot p_{T_D}(D.pre > m \wedge D.post < n) \tag{7}$$

In this equation, we express the AD pattern nodes $T_A$ and $T_D$ separately and use function $p(c)$ denotes the exact selectivity for a predicate $c$. Note that the ancestor node $T_A$ includes both the value join and structural join predicates. We can now express the join selectivity $\Phi(Q)$ in Equation 2 by summing over all keys $v \in A$, and all tree encoding $m, n \in N^*$. Given a relational sample $S_1$, an ancestor sample $S_2$, and a descendant sample $S_3$, we can compute and multiply the individual contributions for all combinations of sample points, and sum over the products. In specific, the join selectivity estimator denoted by $\hat{\Phi}(Q)$ is defined as follow:

$$\hat{\Phi}(Q) = \frac{1}{s_1 \cdot s_2 \cdot s_3} \sum_{v \in R.A \cap T.A} \sum_{m,n \in N^*} \left( \sum_{i=1}^{s_1} \hat{p_R}^{(i)}(A.v = v) \right) \cdot \left( \sum_{j=1}^{s_2} \sum_{k=1}^{s_3} \left( \hat{p_{T_A}}^{(j)}(A.v = v) \right) \cdot \left( \hat{p_{T_A}}^{(j)}(A_{(pre<m,post>n)}) \cdot \hat{p_{T_D}}^{(k)}(D_{(pre>m,post<n)}) \right) \right) \tag{8}$$

where $\hat{p_R}^{(i)}$, $\hat{p_{T_A}}^{(j)}$ and $\hat{p_{T_D}}^{(k)}$ are the individual contribution from $i$-th table sample, j-th ancestor node sample, and k-th descendant node sample, respectively; $s_1$, $s_2$, $s_3$ denote the sample size; v is the sample's join value; m and n are two integer numbers; The *pre* and *post* represent the node's numeric *pre-order* and *post-order* ranks in tree traversal, respectively. Recall the motivating example, $A$ corresponds to id and $v$ is the id value, the ancestor and descendant nodes correspond to the customer and item samples.

One problem an acute reader would immediately notice is that, the estimator presented above is not a closed-form expression as the join value domain of A and the location domain are potentially huge and thus infeasible to be used for computing the join contribution. Instead, we have to exploit properties of the join samples to avoid explicitly summing over the entire join domain. In the next, we demonstrate how to derive a KDE for a RTJSE:

Intuitively, we can compute the value join and structral join independently and then multiply their join contributions:

$$\hat{\Phi}(Q) = \frac{1}{s_1 \cdot s_2} \left( \sum_{i=1}^{s_1} \sum_{j=1}^{s_2} \underbrace{\sum_{v \in R.A \cap T.A} \hat{p_R}^{(i)}(A.v = v) \cdot \hat{p_{T_A}}^{(j)}(A.v = v)}_{J^{\theta}_{i,j}} \right) \cdot \frac{1}{s_2 \cdot s_3} \left( \sum_{j=1}^{s_2} \sum_{k=1}^{s_3} \sum_{pre,post \in N^*} \cdot \left( \hat{p_{T_A}}^{(j)}(A_{(pre<m,post>n)}) \cdot \hat{p_{T_D}}^{(k)}(D_{(pre>m,post<n)}) \right) \right) \tag{9}$$

We refer to $J^{\theta}_{i,j}$ as the cross-model value join contribution. By leveraging a closed-form approximation [15] with a Gaussian kernel to derive the KDE contribution, it can be computed as follow:

$$J^{\theta}_{i,j} = \mathcal{N}_{r^{(i)},(\delta_r^2+\delta_t^2)}(t^{(j)}) \tag{10}$$

where $\mathcal{N}_{\mu,\delta^2}$ denotes the probability density function for a standard normal distribution with mean $\mu$ and variance $\delta^2$, $r^{(i)}$ (resp. $t^{(j)}$) denotes the $i$-th (resp. $j$-th) sample point from R.A (resp. T.A), and $\delta_r$ (resp. $\delta_t$) denotes the estimator bandwidth for R.A (resp. T.A).

Similarly, we also have to exploit properties of the tree samples to avoid the enumeration of the entire location domain. As illustrated in Figure 5a, for a sample ancestor node (marked with a black cross), only the region of its lower right includes the descendant nodes (marked with a red circle) according to the numbering formula. Motivated by this observation, we propose a point-wise method to compute the sample's contribution. Our basic idea is to approximate the contribution by integrating the kernel density over the descendant region of each ancestor, then sum up all the contributions. In particular, we define the *ancestor descendant region* (ADR) of a point p as:

$$ADR(p) = \{q | p.pre < q.pre \wedge p.post > q.post, \forall 1 < pre < u_s, 1 < post < u_s\} \tag{11}$$

where *pre* and *post* denote the point's numeric *pre-order* and *post-order* label, respectively; $u_s$ is the maximum bound of both the *pre* and *post* labels.

Without loss of any generality, we replace the probability of location value of each point with a Gaussian kernel given in Equation 4. Combining with Equation 10, we transform the estimator $\hat{\Phi}(Q)$ in Equation 9 to the following Equation:

$$\hat{\Phi}(Q) = \frac{1}{s_1 \cdot s_2} \sum_{i=1}^{s_1} \sum_{j=1}^{s_2} \left( \mathcal{N}_{r^{(i)},(\delta_r^2+\delta_t^2)}(t^{(j)}) \right) \cdot \frac{1}{s_2 \cdot s_3} \sum_{j=1}^{s_2} \sum_{k=1}^{s_3} \left( \int_{ADR_p^{(j)}} \mathcal{N}_{q^{(k)},h^2}(\vec{x}) d\vec{x} \right) \tag{12}$$

where $ADR(p)^{(i)}$ denotes the ancestor descendant region of $j$-th point $p^{(j)}$ that includes a set of descendant points $q$; $h$ denotes the structural join estimator bandwidth.

By integrating the Gauss distribution of each sample over the $ADR_p^{(j)}$ and grouping by the common factor of the sum of ancestor samples $\sum_{j=1}^{s_2}$, the estimator, termed as Location-Value Estimation (LVE) model, arrives at:

$$\hat{\Phi}(Q) = \frac{1}{s_1 \cdot s_2 \cdot s_3} \sum_{j=1}^{s_2} \left( \sum_{i=1}^{s_1} \left( \mathcal{N}_{r^{(i)},(\delta_r^2+\delta_t^2)}(t^{(j)}) \right) \cdot \sum_{k=1}^{s_3} \left( \frac{1}{2} \left[ erf(\frac{u_{pre} - q^{(k)}_{pre}}{\sqrt{2} \cdot h}) - erf(\frac{l_{pre} - q^{(k)}_{pre}}{\sqrt{2} \cdot h}) \right] \cdot \frac{1}{2} \left[ erf(\frac{u_{post} - q^{(k)}_{post}}{\sqrt{2} \cdot h}) - erf(\frac{l_{post} - q^{(k)}_{post}}{\sqrt{2} \cdot h}) \right] \right) \right) \tag{13}$$

where erf : $\mathbb{R} \rightarrow \mathbb{R}$ denotes the *error function*, which is defined as $erf(x) = \frac{2}{\sqrt{\pi}} \int_0^x \exp -t^2 dt$; $u_{pre}$ and $l_{pre}$ (resp. $u_{post}$ and $l_{post}$ ) denote the upper and lower bounds of the ADR region of $j$-th sample point $p^{(j)}$. In particular, $u_{pre}$ is the maximum preorder numbering; $l_{pre}$ is equal to $p^{(j)}_{pre}$; $u_{post}$ is equal to $p^{(j)}_{post}$ and $l_e$ is 0.

**Remarks.** The estimator we give above is an ancestor-based estimator. A descendant-based estimator is also possible if the descendant node has a cross-model join to a relation. This is achieved by integrating the kernel density over the upper left ancestor region of each descendant. If both ends has a cross-model join, one can combine the ancestor-based estimator and descendant-based estimator by multiplying their contribution.

*3.1.1 Relation-Tree Joins with Twig Pattern.* In addition to the binary structural, the LVE model can also be generalized to more complex relation-tree joins containing a twig pattern join tree. A twig pattern join tree here refers to a join tree with branched paths and with more than two node predicates, where each node predicate

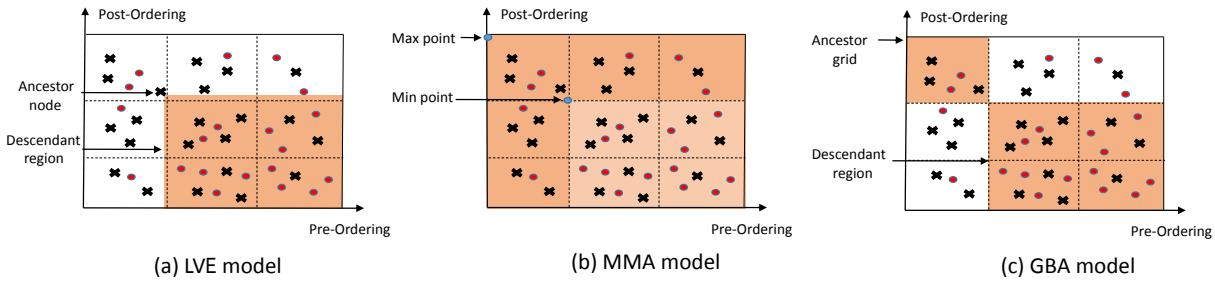



**Figure 5: Location-Based KDE Models**

$p$ in a tree $T$ could be joined with an attribute $a$ in a relation $R$, and the binary structural operators involves the ancestor-descendant (AD) or parent-child (PC) relationship. We have discussed the KDE estimation for AD relationship above. As for the PC relationship, we can simply add a pre-verification step that checks if the ancestor node's level number is exactly 1 less than that of the descendant nodes, then apply the LVE model to the samples.

To apply the LVE model to relation-tree joins with twig patterns, our approach decomposes the twig pattern tree into a sequence of simple binary patterns, then combine the individual estimators. However, there are two cases that lead to the duplicate estimation: (1) the common join ancestor for two binary patterns, and (2) an articulation join node that not only be a descendant node in a binary pattern but also is a ancestor node in another binary pattern. Therefore, we could make the estimation on the value join only once in such cases for the sake of efficiency.

*3.1.2 Bandwidth Optimization.* As we discussed in Section 2.3, the selection of bandwidth has a significant impact on the estimation quality of kernel-based estimators. In this work, we adopt the query-driven bandwidth optimization that has been proved to be a robust and effective approach in optimizing the bandwidth of KDE in previous works [12, 15, 41]. In specific, given a set of representative queries $Q_1, \ldots, Q_n$, we then optimize the bandwidth vectors $h_1, \ldots, h_m$ for the geometric mean over the estimation error [15]:

$$\underset{h_1,\ldots,h_m}{\operatorname{argmin}} \left(\prod_{i=1}^{n} m\mathcal{L}(\Phi_{Q_i}(S'), \Phi_{Q_i}(S))\right)^{\frac{1}{n}} \tag{14}$$

To reduce the learning complexity, we optimize the value join bandwidth $\delta$ and structural-join bandwidth $h$ individually. Specifically, we utilize the stochastic gradient descent (SGD) approach [4] to optimize the bandwidth. Basically, it splits a sample to the training set $S'$ and validation set $S$. Then it computes the estimated join selectivity $\Phi_Q(S')$ of $S'$, and treats the join selectivity $\Phi_Q(S)$ of $S$ as the "true selectivity". The objective is to optimize the bandwidth $h$ such that the loss function $\mathcal{L}$ of $\Phi_Q(S')$ and $\Phi_Q(S)$ is minimized. Through an iterative tuning process that starts with an initial scott value in Equation 5, the bandwidths can at least converge to a local optimal value [41].

## 3.2 Location-Value Estimation (LVE)

Algorithm 1 describes the detailed procedure of Location-Value Estimation (LVE). It takes as input (1) a relation-tree join $Q$, (2) a set of relations $R$ and a set of trees $T$, and (3) the trained KDE model with the value join and structural join bandwidths. In a nutshell, it

**Algorithm 1:** Location-Value Estimation

**Input:** A relation-tree join $Q$, a set of relations $\mathbb{R}$ amd trees $\mathbb{T}$, a sample size $n$ and a trained KDE model *LVE*
**Output:** an estimate based on the *LVE* model

1 $S_\theta \leftarrow$ initialize the visited tree node set for value joins
2 $S_R \leftarrow$ Sample($R$, $n$), $S_T \leftarrow$ Sample($T$, $n$)
3 **foreach** *binary pattern* $P \in Q_Y$ **do**
4  **if** $P_A$ *has a value join with* $R \in \mathbb{R}$ **then**
5  **if** $P_A$ *is not in* $S_\theta$ **then**
6  $\Phi_{P_A} \leftarrow$ compute the join contribution using the ancestor-based estimation in Equation (13)
7  $S_\theta.add(P_A)$
8  **else**
9  $\Phi_{P_A} \leftarrow$ compute only the structural join contribution using the ancestor-based estimation in Equation (13)
10  **else if** $P_D$ *has a value join with* $R \in \mathbb{R}$ **then**
11  **if** $P_D$ *is not in* $S_\theta$ **then**
12  $\Phi_{P_D} \leftarrow$ compute the join contribution using a descendant-based estimation based on Equation (13)
13  $S_\theta.add(P_D)$
14  **else**
15  $\Phi_{P_D} \leftarrow$ compute only the structural join contribution using a descendant-based estimation based on Equation (13)
16  **else**
17  $\Phi_P \leftarrow$ compute only the structural join contribution using the ancestor-based estimation based on Equation (13)
18 **return** $\prod_{k=1}^{n} \Phi_{P_k}$

computes the relation-tree join KDE contribution for each binary pattern $P \in Q_Y$. Then it multiplies the contributions for all binary patterns as the selectivity estimation.

More specifically, LVE works as follows. It first initializes $S_\theta$ with a empty set that keeps the visited tree node for value joins (line 1). In line 2-5, it samples the corresponding relational tuples and tree nodes with sample size $n$. It then performs a KDE estimation for each binary pattern $P \in Q_Y$ (line 6-20). In particular, for a binary pattern $P = (P_A, P_D)$, it checks whether the ancestor pattern node $P_A$ or the descendant pattern node $P_D$ exists a value join with the relations. If it does, it will further check whether they have been computed before, if $P_A \notin S_\theta$ or $P_D \notin S_\theta$, LVE computes the join contribution using Equation 13 based on the ancestor estimation for $P_A$ or the descendant estimation for $P_D$. Otherwise, LVE computes only the structural join contribution based on Equation 13 without

**Algorithm 2:** Max-Min Approximation (MMA)

**Input:** The relational samples $S_R$ and tree samples $S_T$ and a grid $G$.
**Output:** a *KDE* approximator of MMA

1 $J_\theta \leftarrow$ compute the value join contribution based on $S_R$ and $S_T$
2 **foreach** *grid* $g \in G$ **do**
3 $\quad J_u^g, J_l^g \leftarrow$ compute the structural join contribution of max and min points in grid $g$
4 $\quad J_\epsilon = J_\epsilon + (J_u^g - J_l^g)/2$
5 **return** $J_\theta \cdot (J_\epsilon / |g|)$

involving the relational samples. Finally, it returns the selectivity estimation by multiplying all the contributions.

## 4 EFFICIENT KDE APPROXIMATION

Recall the LVE in Section 3.2, we need to make a point-wise KDE to each sample for acquiring the estimates. However, evaluating all the sample points can be prohibitively expensive when sample size increasingly grows. To circumvent this problem, we introduce an $\epsilon$-approximation approach by approximating the point-wise KDE to make a much cheaper estimation. Particularly, we propose two approximation models, max-min approximation (MMA) and grid-based approximation (GBA) models, to improve the estimation efficiency based on the ancestor-based estimation.

### 4.1 Max-Min Approximation (MMA)

The first approximation we propose is the max-min approximation. Its key idea is to derive an $\epsilon$-approximation KDE for a group of ancestor nodes in a grid by just evaluating its *max* and *min* points. As shown in Figure 5b, the *max* and *min* points are two vertices (left upper and right lower) on the left diagonal of a grid. Suppose a grid contains a set of ancestor samples $A$, and the join contribution of *max* and *min* points are $J_u$ and $J_l$, respectively. Obviously, for each sample $a \in A$, the contribution $J_a$ of node $a$ in the grid satisfy $J_l \leq J_a \leq J_u$.

The estimation process of MMA is described in an abridged algorithm 2. Generally, it makes the value join and structural join estimation separately. In line 1, it computes the value join contribution. Line 2-4 computes the average structural join contribution for each grid based on the maximum and minimum point estimation and sum over all the grids. Note that the algorithm requires only $g^2$ times of estimation regardless of the size $n$ of samples, which can significantly improve the efficiency of LVE algorithm when $n >> g$.

Lemma 1. *Given a set of points P, and a grid of size $g \times g$, the method MMA gives an approximate kernel density estimate $KDE_G$ of P, such that, for each binary pattern, we have $|KDE_P - KDE_G| \leq \epsilon$, where $\epsilon$ is equal to $\sum_{i=1}^{g}\sum_{j=1}^{g} \frac{|n_g| \cdot (J_u^{g_{i\times j}} - J_l^{g_{i\times j}})}{2}$.*

Proof 1. *(Sketch) Based on the MMA method, each grid $g_{i\times j}$ has the maximum estimation $J_u^g$ and minimum estimation $J_l^g$ for the structural join, and the value join contribution is same as that of LVE model. Assume an average join contribution $\frac{J_u - J_l}{2}$ for each point, thus the estimation error $\epsilon_g$ for the grid g is no more than $\frac{|n_g| \cdot (J_u - J_l)}{2}$ where $|n_g|$ is the ancestor size of grid g. By summing up all the estimation error for the grid, the Lemma 1 holds.*

**Algorithm 3:** Grid-Based Approximation (GBA)

**Input:** The relational samples $S_R$ and tree samples $S_T$ and a grid $G$.
**Output:** a *KDE* approximator of GBA

1 $J_\theta \leftarrow$ compute the value join contribution based on $S_R$ and $S_T$
2 **foreach** *grid* $g \in G$ **do**
3 $\quad J_\epsilon = J_\epsilon + J_g^A * (J_{G^{g^{lr}}}^D + \frac{1}{16} J_g^D + \frac{1}{4} \cdot (J_{G_g^r}^D + J_{G_g^l}^D)$
4 **return** $J_\theta \cdot (J_\epsilon / |g|)$

**MMA estimator.** Given a grid size $|g|$ and a set of sample points $p$, we derive an MMA estimator by assuming an average error for each point as follow:

$$\hat{\Phi}_{MMA} = J_\theta \cdot \frac{1}{|g|} \cdot \sum_{i=1}^{g}\sum_{j=1}^{g} \frac{(J_u - J_l)}{2} \tag{15}$$

where $J_\theta$ denotes the value join estimator.

### 4.2 Grid-Based Approximation (GBA)

To consider the overlap property of data, e.g. customer nodes can have the descendant customer nodes, we also propose an grid-based approximation (GBA) to improve the efficiency of LVE model. Figure 5c demonstrates the main idea of GBA. The intuition is that since ancestor nodes can have many overlap descendant nodes, we can substantially reduce the number of estimation by grouping the points to grids and then estimating the join contribution for an ancestor grid and its corresponding descendant grids. More precisely, the estimator divides the target region to $g \times g$ grids based on the bounds of pre-ordering and post-ordering, then makes the estimation grid by grid. However, in addition to the diagonal AD case between two regions, i.e., an ancestor grid with its lower-right region, there are three cases also contribute to KDE which we can not neglect: (i) an ancestor grid with its lower region; (ii) an ancestor grid with its right region; (iii) an overlap grid with both ancestor and descendant nodes.

Formally, given a global $g \times g$ grid $G_{g\times g}$ and an arbitrary grid $g_{i\times j} \subset G$ with $i < g$ and $j < g$, we distinguish four cases in contributing to a structural join KDE:

- Case 1: Grid $g$ with its adjacent lower-right region $G^{g^{lr}}$ in G. Obviously, this case gives the exact KDE.
- Case 2: Grid $g$ itself with both ancestor and descendant samples. We denote four evenly partitions as left upper $g^{lu}$, left lower $g^{ll}$, right upper $g^{ru}$, and right lower $g^{rl}$.
- Case 3: Grid $g$ with its adjacent right region $G^{g^r}$ in G.
- Case 4: Grid $g$ with its adjacent lower region $G^{g^l}$ in G.

The estimation process of GBA is described in an abridged algorithm above. Note that line 2-4 essentially uses the uniform assumption [37] to estimate the KDE in case 2-4. For case 1, since it gives the exact KDE, we simply add up its join contribution. The algorithm requires $4 \cdot g^2$ times of estimation regardless of the size $n$ of samples, which can largely improve the efficiency of LVE algorithm.

Lemma 2. *Given a set of points P, and a grid of size $g \times g$, for an arbitrary grid $g_{i\times j}$ ($i < |g|$ and $j < |g|$), the upper gap bound of GBA's KDE $\epsilon_g$ is $J_{g^{lu}}^A \cdot (J_{g^{rl}}^D + J_{G_g^r}^D + J_{G_g^l}^D)$.*

Proof 2. *(Sketch) We can proof Lemma 2 by contradiction. Let $g$ be the maximum case that all the ancestor nodes of A fall in $g^{lu}$, and all the descendant nodes of D fall in $g^{rl} \vee G^{g^{rl}} \vee G^{g^{lr}}$. Assume $g'$ is the outlier case that all ancestor nodes fall in $g'^{lu}$ but one point $x$ falls in a region $r \not\subset g'^{lu}$. If $x \in g'^{ll}$ or $x \in g'^{ru}$ or $x \in g'^{lr}$, then $J^A_{g^{lu}} > J^A_{g'^{lu}}$. On the other hand, if any descendant node $y$ falls in a region $r \not\subset (g^{rl} \vee G^{g^r} \vee G^{g^l})$, we have $J^A_{g^{lu}} \cdot J^D_g > J^A_{g^{lu}} \cdot J^D_r$. That concludes the proof.*

**GBA estimator.** Given a grid size $|g|$ and a set of sample points $p$, we derive an GBA estimator by assuming a uniform join contribution [37] for each grid as follow:

$$\hat{\Phi}_{GBA} = J_\theta \cdot \frac{1}{|g|} \sum_{i=1}^{g} \sum_{j=1}^{g} J^A_g * (J^D_{G^{g^{lr}}} + \frac{1}{16} J^D_g + \frac{1}{4} \cdot (J^D_{G^r_g} + J^D_{G^l_g})) \quad (16)$$

# 5 EXPERIMENT

In this section, we present the experimental evaluation of our KDE-based join estimators in terms of estimation quality, execution efficiency, and scalability. In particular, we implement all experiments in Python v3.6. We adopt the built-in KDE model provided by the `Scipy` library [14]. We run all experiments on a machine with a 4-core i5-4590 CPU and 16GB RAM, and 1TB HDD.

## 5.1 Compared Estimator

We compared the following six estimators:

- **Simple Random Sampling (SRS):** Our first baseline estimator uses the simple random sampling that implements naive sample evaluation based on uniform samples from the base tables and tree encoding. We leverage a hash join to speed up the computation of join estimation.
- **Position Histogram (PH):** The second compared estimator is the Position Histogram [37] that employed histograms as statistical summaries over the tree data to estimate the structural join size. In terms of the value joins, we adopt the SRS estimator to derive a separate join estimates, then combine it to the final estimates.
- **Join KDE (RJ-KDE)**: The third estimator is the state-of-the-art Join KDE [15] that was originally proposed to estimate the relational join selectivity. We extend the join estimator to estimate the structural join selectivity by incorporating the SRS structural estimator.
- **Location-Value Estimation KDE (LVE-KDE)**: The fourth estimator is our Location-Value (LV) KDE estimator introduced in Section 3. Similar to RJ-KDE, We adopt the pruning strategies [15] to neglect those tuples that have sufficiently small join contributions.
- **Max-Min KDE Approximation (MMA-KDE)** and **Grid-Based KDE Approximation(GBA-KDE)**: The last two estimators are our KDE approximators based on the MMA and GBA methods introduced in Section 4.

**Remarks.** Note that for the SRS estimator, the tree data has been encoded and the estimation is made by computing the join size from samples of relational data and tree encoding. SRS consists of the relational estimator and structural estimator, either of which can be combined with other estimators to derive an independent relation-tree join estimate.

## 5.2 Evaluated Datasets

*Synthetic dataset generator.* We created a light-weight multi-model data generator to provide the synthetic data with both structural and value dependencies. In particular, it works as follows. First, it inputs a seed dataset with a given `Zipfian` skewness parameter $\alpha$ that is used to control the join selectivity. The generator preserves the original structural dependency of dataset and collects the seed join values from the corresponding tree information, i.e., attributes value, child value or content value. It then re-assign the seed join values to the join nodes with skewness $\alpha$. Finally, it generates the corresponding join tables with the identical $\alpha$. In this work, we generate the values of primary key for a table.

*Seed dataset.* We employed both synthetic and real-life datasets as the seed datasets which includes two tree datasets and two graph datasets. The tree datasets includes the `DBLP`[1] and `XMARK` [30] datasets. The DBLP dataset is a well-known bibliography that provides the information of publication and co-authorship in the field of computer science. The XMARK dataset [30] is a synthetic dataset that mimics a Internet auction site in a tree-structured schema. The graph datasets are the `LDBC`[2] dataset and `UniBench` [39] datasets. The `LDBC` dataset is a scalable synthetic social network dataset and the UniBench dataset is a multi-model social commerce dataset. In particular, we utilize the `NetworkX`[3] library to generate the join tree from the graphs by acquiring a spanning tree with a depth limit 10 from the graph so that the join tree contains the overlap property. We say a node predicate has overlap property, meaning that its descendants can have the same predicate.

## 5.3 Evaluated Workloads

As illustrated in Table 1, we have designed 12 relation-tree joins with structural and equality predicates. For the synthetic relations, each relation has the form: `relation name (join key)`, with 1 million tuples. For the tree pattern, we consider the different types of structural joins, including the AD relationship, path-based query, and twig queries. For instance, Q12 is a relation-tree join with a twig query that finds persons who have bought products and given the feedback. As a result, it involves two AD relationships i.e. person//product and person//feedback, and three relation-tree value joins. For the tree data generated from two graph datasets, i.e., LDBC and UniBench, predicate *person* has the overlap property, i.e., person knows person. In the last column, we also show the result size of each twig pattern.

## 5.4 Evaluation of Estimation Accuracy

In the first set of experiments, we compared the accuracy of all the estimators in Section 5.1. We fixed the data skewness $\alpha$ to 1 and set the sample size to a small portion of the data, i.e., 100. The bandwidths for three KDE models we optimized have a small training error. We set the grid size for all grid-based models to 30.

[1] https://dblp.org/xml/release/
[2] https://github.com/ldbc/ldbc_snb_datagen
[3] https://networkx.github.io/

**Table 1: Statistics for Datasets and Descriptions for Relation-Tree Joins**

| Dataset | Statistics | Query | Synthetic Relations | Tree Pattern | Twig Size |
|---|---|---|---|---|---|
| DBLP | Data size: 127 MB | Q1: | R1(inproceedings), R2(author) | //inproceedings//author | 491783 |
| | Tree nodes: 33 million | Q2: | R1(article), R2(journal) | //article//journal | 111609 |
| | relation nodes: 2 million | Q3: | R1(article), R2(title) | //article[.//sup]//title//sub | 351 |
| XMARK | Data size: 111 MB | Q4: | R1(person), R2(watch) | //person//watch | 50269 |
| | Tree nodes: 16 million | Q5: | R1(open_auction), R2(itemref) | //open_auction//itemref | 12000 |
| | relation nodes: 2 million | Q6: | R1(regions), R2(item) | //regions//item/location | 21750 |
| LDBC | Data size: 1.16 GB | Q7: | R1(person), R2(comment) | //person//comment | 12121 |
| | Tree nodes: 3 million | Q8: | R1(person), R2(post) | //person//post | 13574 |
| | relation nodes: 2 million | Q9: | R1(person), R2(comment), R3(post) | //person//[.//comment]//post | 9875 |
| UniBench | Data size: 3.52 GB | Q10: | R1(person), R2(product) | //person//product | 3425 |
| | Tree nodes: 3 million | Q11: | R1(person), R2(feedback) | //person//feedback | 2347 |
| | relation nodes: 2 million | Q12: | R1(person), R2(product), R3(feedback) | //person[.//product]//feedback | 1263 |

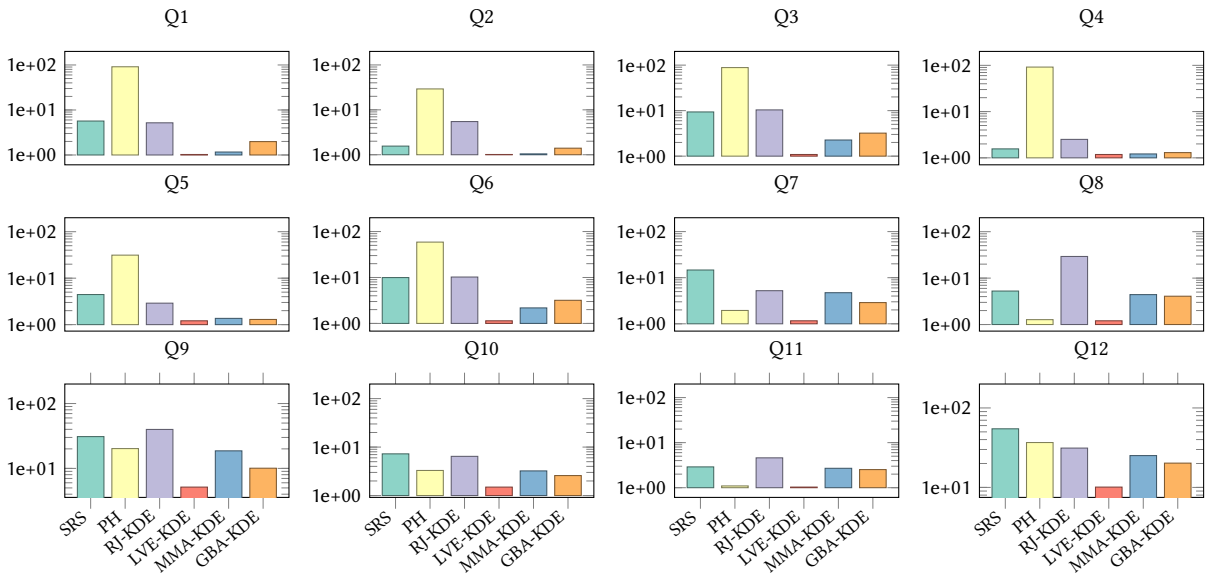



**Figure 6: Estimation Accuracy of compared estimators. The y-axis shows the multiplicative estimation error.**

We ran the experiment with 10 different sample sets and report the median results of the multiplicative error that is calculated by the equation: $m(J, \hat{J}) = \frac{max(J,\hat{J})}{min(J,\hat{J})}$, where $J$ is the true selectivity and $\hat{J}$ is the estimated join selectivity. Note that when the estimate $\hat{J}$ is larger than the true selectivity $J$, the multiplicative error is $\hat{J}/J$, otherwise it is the $J/\hat{J}$. Thus, the multiplicative error equally penalize over- and underestimations and its smallest value is 1.0.

The results in Figure 6 clearly shows that LVE-KDE is the best estimator in all relation-join cases. We attributes this to its KDE generalization over the samples with bandwidth optimization, which result in a more robust and accurate estimate. In addition, two proposed KDE approximators, i.e., MMA-KDE and GBA-KDE, offer superior estimation quality in the DBLP dataset (Q1-Q3) and XMARK dataset (Q4-Q6) despite they allows some failure probability under the uniform assumption. An interesting observation is that MMA and GBA are complementary, in the sense that MMA tends to be better in the non-overlap cases (Q1-Q6), while GBA is better in the overlap cases (Q7-Q12) because of the grid-based estimation. Nevertheless, both approximators are no worse than the naive sampling, SRS, in all twelve joins, which means these two methods are *worthwhile* as they uses the same sample size as that of SRS method.

We find the histogram-based approach PH-histogram usually overestimates the selectivity up to two orders of magnitude in the unselective cases (Q1-Q6), while it offers good estimates in the selective cases (Q7-Q12). The RJ-KDE achieves good accuracy in estimating the equi-join selectivity, however, it incurs higher error after incorporating the SRS structural estimator. In particular, it is worse than pure SRS estimator in three cases, i.e., Q8, Q9, Q11. The reason is that it uses the separate estimator for estimating the structural join and value join contribution.

## 5.5 Evaluation of Estimation Efficiency

The second set of experiments is to evaluate the efficiency of different estimators. For this, we perform our previous experiments

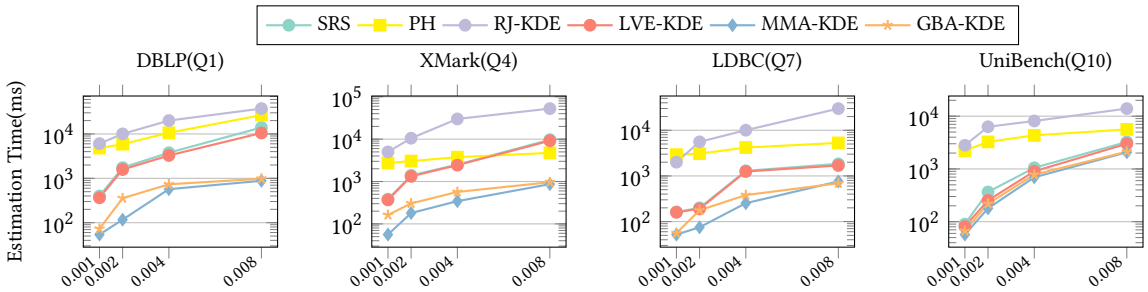



**Figure 7: Estimation Efficiency vs. Sample ratio. The x-axis shows the sample ratio.**

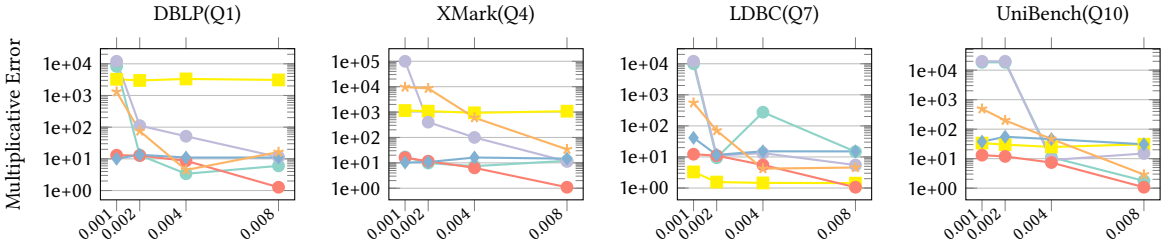



**Figure 8: Estimation Accuracy vs. Sample ratio. The x-axis shows the sample ratio.**

while increasing the sample ratio by factors of two from 0.001 to 0.008. We report the estimation time for a representative query for each dataset, namely, Q1, Q4, Q7, Q10 over the DBLP, XMARK, LDBC, and UniBench datasets, respectively.

Figure 7 illustrates the running time for estimating the selectivity of chosen queries. The first observation is that the LVE-KDE estimator basically performed the same as other competitors. This is mainly attributed to our sample pruning strategies that significantly reduce the number of computations required to compute an estimate. Besides, each sample point computes the KDE contribution of structural join just once based on its location. Second, its variants, i.e., MMA-KDE and GBA-KDE, achieved better efficiency in estimating the selectivity. This is mainly because they do not enumerate all sample points. Moreover, they can return the result in milliseconds, which generally outperform RJ-KDE by 1 to 2 orders of magnitude. The reason is that RJ-KDE computes the structural join contribution by enumerating every combination from the data. The final observation is that the MMA-KDE and GBA-KDE can speed up the LVE-KDE in Q1 and Q4 with a factor of 4 and 2, respectively. But they have little benefit in relative small trees, i.e., Q7 and Q10. Thus, they can serve as the candidate estimators when the LVE-KDE faces large tree samples.

## 5.6 Evaluation of Estimation Scalability

The third set of experiments is to evaluate the scalability of different estimators. The goal is to understand how the estimators perform as the sample size increases. Similar to Section 5.5, we report the accuracy while increasing the sample ratio by factors of two from 0.001 to 0.008.

The results are shown in Figure 8. We first observe that for sample ratio smaller than 0.002, RJ-KDE and SRS methods have a larger estimation error due to the scanty join size in the samples. As the sample ratio increases up to 0.008, they dramatically improve the estimation error. Second, the PH method overestimates the selectivity by orders of magnitude in Q1, Q4 and Q10. Besides, it makes little improvement as the sample size grows. Again, this is because of the grid estimation based on the uniform assumption. Our key observation is that LVE-KDE can provide best estimates in all four cases when the ratio is close to 0.008, which confirms LVE-KDE can significantly improve the estimates computed from samples. Moreover, LVE-KDE is robust as it uses a unified KDE framework, which outperforms others in the case of sample ratio smaller than 0.002. An exception is Q7, where the PH is the winner because of the uniform distribution of data. We also find that the GBA-KDE and MMA-KDE can provide the KDE approximation error of LVE-KDE with around a factor of 10, which is generally no worse than PH and close to SRS when the ratio reaches 0.008.

## 5.7 Evaluation of Grid-based Estimators

In order to further investigate the impact of grid size on three grid-based estimators, namely, PH, GBA-KDE, and MMA-KDE, we repeat the previous experiments with sample ratio 0.02. We evaluate both the accuracy and efficiency by varying the grid size with prime number ranging from 1 to 11, with a multiplicative factor of 3.

Figure 9 reports the results for the accuracy evaluation. We observe that the grid size has significant impact on the estimation quality of three grid-based estimators, which can make remarkable difference to the estimates by orders of magnitude. Interestingly, the impact of grid size is similar to that of bandwidth. That is, choosing a small grid size overestimates the join selectivity, while choosing a large grid size underestimates the selectivity. For instance, GBA-KDE has the minimum error for Q1 and Q7, with grid size 39 and 3, respectively. In practice, we can use a Monte-Carlo Simulation [17] to guide the selection of grid size for overlap and non-overlap cases. The second observation is that GBA-KDE and MMA-KDE

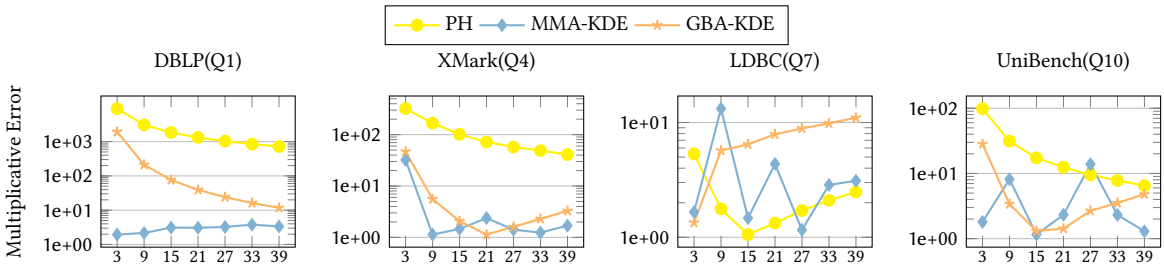



**Figure 9: Estimation Accuracy vs. Grid Size. The x-axis shows the grid size.**

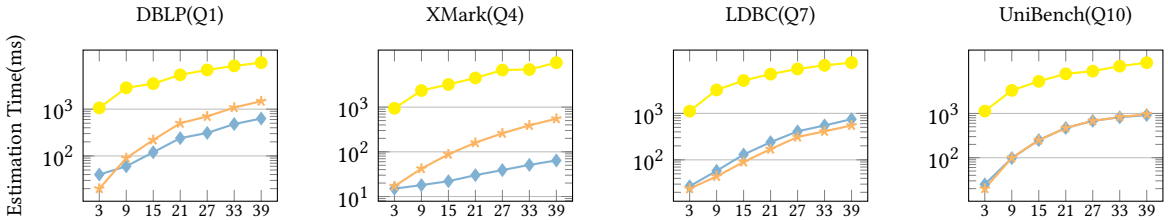



**Figure 10: Estimation Efficiency vs. Grid Size. The x-axis shows the grid size.**

converge much faster than PH as the grid size increases, verifying the statement that KDE converges faster than histogram do.

Figure 10 illustrates the results for the efficiency evaluation. It is clearly visible that MMA-KDE and GBA-KDE are 1-2 orders of magnitude faster than PH methods in all cases. In particular, both KDE-based approaches can provide the estiamtes in milliseconds under the grid size 33. In contrast, PH has a substantial overhead when grid size reaches 39.

## 5.8 Evaluation Summary

Based on the results of our experiments, we summarize the gained insights as follows:

1. Our LVE-KDE join estimator signifcantly outperform other estimators in accuracy and efficiency. Particularly, LVE-KDE improves 80%, 82%, 96% of estimation accuracy compared with RJ-KDE, SRS, and PH, respectively. Moreover, when the sample ratio is close to one percent of the data, LVE-KDE is roughly 5.1×, 1.2× faster than PH and SRS respectively, and faster than RJ-KDE by an order of magnitude.
2. Our two KDE approximators can 4× and 2× speed up the LVE-KDE in large tree dataset, respectively. Regarding the estimation accuracy, MMA-KDE improves 65%, 67%, 93% of accuracy over RJ-KDE, SRS, and PH, respectively. GBA-KDE improves 70%, 71%, 94% of accuracy over RJ-KDE, SRS, and PH, respectively. Particularly, MMA is the better in the case of non-overlap queries, whereas GBA is the winner in the overlap cases.

# 6 RELATED WORK AND DISCUSSIONS

## 6.1 Related Work

**Sampling-based Estimator:** There has been a host of work on relational join selectivity estimation using sampling-based approaches, (e.g. [5, 11, 18, 34]). The work in [11] studied the random sampling for estimating the join selectivity. In [34], correlated sampling was proposed to provide an estimate of the join size by considering the correlation of tuples in multiple relations. Further, [5] proposed a two-level sampling that combines the advantages of independent Bernoulli sampling, end-biased sampling [7], and correlated sampling. In addition to the relational join, there has also been work on sampling for structural join (e.g. [22, 36]), Wang. et al.[36] proposed an adaptive sampling to estimate the containment join. In [22], a subtree sampling was developed to estimate the twig query selectivity. Our work also involves a sampling process that samples relational tuples and tree nodes to build the KDE model.

**Histogram-based Estimator:** Histogram-based approaches [9] are widely in the context of range query optimization in relational databases. There are also several works on the structural join in tree data [36, 37]. In particular, Wu. et al. [37] employed a numbering schema with a start and a end label to encode the tree and then used the PH-histogram to estimate the selectivity of the binary structural join, i.e., AD and PC relationship. In [36], two one-dimensional models, namely, the interval model and the position model, were designed to encode the AD tree relationship, then they applied a histogram approach to them for obtaining the structural join estimates. To the best of our knowledge, our work is the first work that applies the KDE to selectivity estimation of relation-tree joins.

**KDE-based Estimator:** KDE is a widely used method in statistic and database field [12, 15, 41, 42]. Kiefer et al. utilized the KDE models to estimate the selectivity of relational range query [12] and join query [15]. Particularly, they implemented the KDE estimators using graphics card as GPU programs to speed up the KDE computation. In [41], a KDE-based estimator was proposed to estimate the cardinality of skyline query. Zhang et al. [42] studied the approximations for KDE in large data. This paper differentiates from previous KDE-based works in that we apply the KDE model to the RTJSE problem, which essentially results in a new KDE estimator.

## 6.2 Discussions

**KDE on data updates**: one major concern for KDE is that the data updates have an impact on the KDE estimation quality. In such cases, we can adopt an incremental method from [12] to react to the data updates. For new inserted data, it uses a reservoir sampling algorithm [35] to add new data to the sample with probability $|S|/|R|$, without replacement. For deletions and updates, it leverages the Karma-based maintenance method to replace the outdated samples that affect the estimation quality.

**Integrate existing platforms with KDE:** embedding a KDE estimator into the existing multi-model platforms is practical. Heimel et al. [13] have done the first work that integrated the KDE to the PostgreSQL for relational selectivity estimation. In the future, we plan to extend our KDE estimator to relation-tree joins in PostgreSQL as well. As suggested in [15], GPU-accelerated KDE models can provide a satisfactory estimation quality of join selectivity with 10 ms, we can also implement our KDE model using GPUs.

# 7 CONCLUSIONS

This paper studies a new problem of estimating the join selectivity of relation-tree joins. We propose the LVE model, a kernel-based approach to deal with the estimation problem by considering both value joins and structural joins. Furthermore, we propose two approaches, MMA and GBA models, to approximate the KDE contribution of LVE model while improving the estimation efficiency for large data samples. Extensive experiments confirm that LVE model and its variants achieve high accuracy and efficiency in a variety of real and synthetic datasets.

# ACKNOWLEDGMENTS

This work is partially supported by Academy of Finland (310321) and China Scholarship Council fellowship (201606360137).

# REFERENCES

[1] Noga Alon, Phillip B Gibbons, Yossi Matias, and Mario Szegedy. 2002. Tracking join and self-join sizes in limited storage. *J. Comput. Syst. Sci.* 64, 3 (2002), 719–747.
[2] Björn Blohsfeld, Dieter Korus, and Bernhard Seeger. 1999. A Comparison of Selectivity Estimators for Range Queries on Metric Attributes. In *SIGMOD*. 239–250.
[3] Carlyna Bondiombouy and Patrick Valduriez. 2016. Query processing in multistore systems: an overview. (2016).
[4] Léon Bottou. 2010. Large-scale machine learning with stochastic gradient descent. In *COMPSTAT*. Springer, 177–186.
[5] Yu Chen and Ke Yi. 2017. Two-Level Sampling for Join Size Estimation. In *SIGMOD*. 759–774.
[6] Orri Erling, Alex Averbuch, Josep Larriba-Pey, Hassan Chafi, Andrey Gubichev, Arnau Prat, Minh-Duc Pham, and Peter Boncz. 2015. The LDBC social network benchmark: Interactive workload. In *SIGMOD*. ACM, 619–630.
[7] Cristian Estan and Jeffrey F Naughton. 2006. End-biased samples for join cardinality estimation. In *ICDE*. 20–20.
[8] Edgar Tello-Leal et al. 2016. A Technological Solution to Provide Integrated and Process-Oriented Care Services in Healthcare Organizations. *IEEE Trans. Industrial Informatics* 12, 4 (2016), 1508–1518.
[9] Filippo Furfaro, Giuseppe M Mazzeo, Domenico Saccà, and Cristina Sirangelo. 2008. Compressed hierarchical binary histograms for summarizing multidimensional data. *Knowledge and information systems* 15, 3 (2008), 335–380.
[10] Torsten Grust, Maurice Van Keulen, and Jens Teubner. 2003. Staircase join: Teach a relational DBMS to watch its (axis) steps. In *VLDB*. Elsevier, 524–535.
[11] Peter J. Haas, Jeffrey F. Naughton, S. Seshadri, and Arun N. Swami. 1996. Selectivity and Cost Estimation for Joins Based on Random Sampling. *J. Comput. System Sci.* 52, 3 (1996), 550–569.
[12] Max Heimel, Martin Kiefer, and Volker Markl. 2015. Self-Tuning, GPU-Accelerated Kernel Density Models for Multidimensional Selectivity Estimation. In *SIGMOD*. 1477–1492.
[13] Max Heimel and Volker Markl. 2012. A First Step Towards GPU-assisted Query Optimization. *ADMS@ VLDB* 2012 (2012), 33–44.
[14] Eric Jones, Travis Oliphant, Pearu Peterson, et al. 2001. SciPy: Open source scientific tools for Python. http://www.scipy.org/
[15] Martin Kiefer, Max Heimel, Sebastian Breß, and Volker Markl. 2017. Estimating Join Selectivities using Bandwidth-Optimized Kernel Density Models. *PVLDB* 10, 13 (2017), 2085–2096.
[16] Andreas Kipf, Thomas Kipf, Bernhard Radke, Viktor Leis, Peter A. Boncz, and Alfons Kemper. 2019. Learned Cardinalities: Estimating Correlated Joins with Deep Learning. (2019).
[17] David P Landau and Kurt Binder. 2014. *A guide to Monte Carlo simulations in statistical physics*. Cambridge university press.
[18] Viktor Leis, Bernhard Radke, Andrey Gubichev, Alfons Kemper, and Thomas Neumann. 2017. Cardinality Estimation Done Right: Index-Based Join Sampling.. In *CIDR*.
[19] Zhen Hua Liu, Beda Hammerschmidt, and Doug McMahon. 2014. JSON data management: supporting schema-less development in RDBMS. In *SIGMOD*. 1247–1258.
[20] Jiaheng Lu and Irena Holubová. 2017. Multi-model Data Management: What's New and What's Next?. In *EDBT*.
[21] Jiaheng Lu and Irena Holubová. 2019. Multi-model Databases: A New Journey to Handle the Variety of Data. *ACM Comput. Surv.* 52, 3 (2019), 55:1–55:38.
[22] Cheng Luo, Zhewei Jiang, Wen-Chi Hou, Feng Yu, and Qiang Zhu. 2009. A sampling approach for XML query selectivity estimation. In *EDBT*. 335–344.
[23] J Melton. 2003. Information technology–database languages–SQL–Part 14: XML-related specifications (SQL/XML). *ISO/IEC* (2003), 9075–14.
[24] Chaitanya Mishra and Nick Koudas. 2009. Interactive query refinement. In *EDBT (ACM International Conference Proceeding Series)*, Martin L. Kersten, Boris Novikov, Jens Teubner, Vladimir Polutin, and Stefan Manegold (Eds.), Vol. 360. ACM, 862–873.
[25] Oracle. 2019. Database JSON Developer's Guide. https://docs.oracle.com/en/database/oracle/oracle-database/12.2/adjsn/store-and-manage-json-data.html
[26] Neoklis Polyzotis, Minos Garofalakis, and Yannis Ioannidis. 2004. Selectivity estimation for XML twigs. In *ICDE*. 264–275.
[27] Zeenat Rehena and Marijn Janssen. 2018. Towards a Framework for Context-Aware Intelligent Traffic Management System in Smart Cities. In *WWW*. 893–898.
[28] Florin Rusu and Alin Dobra. 2008. Sketches for size of join estimation. *TODS* 33, 3 (2008), 15.
[29] Mohamed Sarwat, Raha Moraffah, Mohamed F. Mokbel, and James L. Avery. 2017. Database System Support for Personalized Recommendation Applications. In *ICDE*. 1320–1331.
[30] Albrecht Schmidt, Florian Waas, Martin Kersten, Michael J Carey, Ioana Manolescu, and Ralph Busse. 2002. XMark: A benchmark for XML data management. In *VLDB*. Elsevier, 974–985.
[31] David W Scott. 2015. *Multivariate density estimation: theory, practice, and visualization*. John Wiley & Sons.
[32] Arun N. Swami and K. Bernhard Schiefer. 1994. On the Estimation of Join Result Sizes. In *EDBT*. 287–300.
[33] Kostas Tzoumas, Amol Deshpande, and Christian S Jensen. 2011. Lightweight graphical models for selectivity estimation without independence assumptions. *PVLDB* 4, 11 (2011), 852–863.
[34] David Vengerov, Andre Cavalheiro Menck, Mohamed Zaït, and Sunil Chakkappen. 2015. Join Size Estimation Subject to Filter Conditions. *PVLDB* 8, 12 (2015), 1530–1541.
[35] Jeffrey S Vitter. 1985. Random sampling with a reservoir. *TOMS* 11, 1 (1985), 37–57.
[36] Wei Wang, Haifeng Jiang, Hongjun Lu, and Jeffrey Xu Yu. 2003. Containment Join Size Estimation: Models and Methods. In *SIGMOD*. 145–156.
[37] Yuqing Wu, Jignesh M Patel, and HV Jagadish. 2003. Using histograms to estimate answer sizes for XML queries. *Information Systems* 28, 1-2 (2003), 33–59.
[38] Chao Zhang and Jiaheng Lu. 2019. Holistic Evaluation in Multi-Model Databases Benchmarking. *Distributed and Parallel Databases* (2019).
[39] Chao Zhang, Jiaheng Lu, Pengfei Xu, and Yuxing Chen. 2018. UniBench: A Benchmark for Multi-Model Database Management Systems. In *TPCTC*. Springer, 7–23.
[40] Ning Zhang, M Tamer Ozsu, Ashraf Aboulnaga, and Ihab F Ilyas. 2006. Xseed: Accurate and fast cardinality estimation for xpath queries. In *ICDE*. 61–61.
[41] Zhenjie Zhang, Yin Yang, Ruichu Cai, Dimitris Papadias, and Anthony K. H. Tung. 2009. Kernel-based skyline cardinality estimation. In *SIGMOD*. 509–522.
[42] Yan Zheng, Jeffrey Jestes, Jeff M. Phillips, and Feifei Li. 2013. Quality and efficiency for kernel density estimates in large data. In *SIGMOD*. 433–444.